Resolver

James Augustus

Resolver is the unit that converts a number's normal form to shadow: it finds the exponents of seven small primes (2, 3, 5, 7, 11, 13, 17) and the residual not divisible by them. The shadow system can both keep a number as a binomial, such as $19 = 2^4 + 3$ in the form $2^k \pm r$, and factor it. This is a feature of one of the processors I am designing, and this processor can also do base arithmetic. Here I explain the forms that have been coded and tested, from newest to oldest. Forms not implemented in code or left only as examples are in Resolver experiments. Each form gives its date and source, idea, mechanism, an example verified by a program, the state of the code and testing, and its difference from the previous form. Numbers are decimal unless marked with $_8$.

Order	Form	Date	Testing
1	Flowing counter	12-13 October 2026	C core, every number up to 2^{24}
2	Flow folding, modulo $2^p - 1$	13 September 2026	remainder calculation with six different p values
3	Folding gate and exponent-positional shadow	9 September 2026	393,216 results
4	Digit weights and cutting	20 July 2026	weights up to 2^{24}
5	First version: two-beat pipeline	30 June 2026	about 117 million checks

Contents

Section	Title
1	Flowing counter
1.1	Date and source
1.2	Idea
1.3	Mechanism
1.3.1	Remainder flow
1.3.2	Quotient flow
1.3.3	Octal exponent counter
1.4	Example
1.5	Code and testing
1.6	Difference
2	Flow folding, modulo $2^p - 1$
2.1	Date and source
2.2	Idea
2.3	Mechanism
2.4	Example

Section	Title
2.5	Code and testing
2.6	Difference
3	Folding gate and exponent-positional shadow
3.1	Date and source
3.2	Idea
3.3	Mechanism
3.4	Example
3.5	Code and testing
3.6	Exponent-positional shadow
3.7	Difference
4	Digit weights and cutting
4.1	Date and source
4.2	Idea
4.3	Mechanism
4.3.1	Cutting
4.4	Example
4.5	Code and testing
4.6	Difference
5	First version: two-beat pipeline
5.1	Date and source
5.2	Idea
5.3	Mechanism
5.3.1	First beat, divisibility
5.3.2	Second beat, finding exponents
5.4	Example
5.5	Code and testing
5.6	Difference
6	Open questions

1. Flowing counter

1.1 Date and source

12-13 October 2026. Description: “The number came in and was split into parts: as Horner and as a different base. It is kept as data and is simple to resolve.” This description became a single-file C core.

1.2 Idea

The number starts being divided as it enters the machine. As the digits flow from left to right, both the remainder and the quotient come out of the same flow. The quotient flows to the next stage and is divided again there. Exponents accumulate in an octal odometer.

1.3 Mechanism

There are three parts.

1.3.1 Remainder flow

As each digit d arrives, seven channels update $r_q \leftarrow (8\,r_q + d) \bmod q$ at the same time. The moment the last digit enters, all seven remainders are ready. For $q = 7$, since $8 \equiv 1$, the remainder is the sum of the octal digits modulo 7. For $q = 3$, since $8 \equiv -1$, the digits are alternately added and subtracted. For $q = 2$, only the last digit is read.

1.3.2 Quotient flow

The quotient digits also come out in the same flow: $t = 8\,r + d$, quotient digit $\lfloor t/q \rfloor$, new remainder $t \bmod q$. The quotient digits enter the next stage like a conveyor belt.

1.3.3 Octal exponent counter

Each division increases the channel's counter by one. Four rows are the octal digits of one exponent: $e = rs1 + 8\,rs2 + 64\,rs3 + 512\,rs4$, at most 4095. If a row turns from 7 to 0, the carry goes to the upper row.

1.4 Example

$3^{14} = 4\,782\,969 = 22175571_8$. In the remainder flow, only channel 3 reaches 0. In the quotient flow, the value flows through each stage divided by 3. On the eighth division, the counter turns from 7 to 0 and the carry goes to the second row. On the fourteenth division, the value becomes 1. The exponent read is $e_3 = 6 + 8 \cdot 1 = 14 = 16_8$. I show the same mechanism digit by digit and frame by frame with other numbers in the gear model.

1.5 Code and testing

C99 core. Rerun on 25 October 2026: conversion and reconstruction for every number in $1 \le N \le 2^{24}$ gave 0 errors. Addition, subtraction, multiplication and division on 100,000 random pairs of up to 200 octal digits gave 0 errors. Errors were caught in all three deliberately broken copies.

1.6 Difference

In the layered ladder in Resolver experiments, the shadow rows were the levels of the ladder and an exponent cell held at most 7. Here the same four rows are the octal digits of a single exponent. What is carried down is the digit of the exponent, not the remaining value.

2. Flow folding, modulo $2^p - 1$

2.1 Date and source

13 September 2026. Studies 24 and 26 in the Mersenne research line.

2.2 Idea

The remainder modulo $M = 2^p - 1$ is found without division. Since $2^p \equiv 1 \pmod{M}$, the part of the number above bit p can be folded down and added to the lower part. As a very

long number flows in multiple pieces (limbs), each piece is shifted to the right position and added to the same ring.

2.3 Mechanism

$$V \leftarrow (V \gg p) + (V \& M), \qquad V = M \text{ then } V \leftarrow 0$$

This repeats until the value fits in p bits. This is a ring: the bit overflowing at the top wraps to the bottom. In the flow version, piece i is shifted to position $(i \cdot w) \bmod p$ and added to the adder.

2.4 Example

$p = 13$, $M = 8191$, $V = 37634$:

$$37634 = 4 \cdot 2^{13} + 4866 \quad \Rightarrow \quad 4 + 4866 = 4870$$

Indeed, $37634 \bmod 8191 = 4870$.

2.5 Code and testing

Python. Tested with $p = 13, 31, 61, 127, 521$ and 4096, with different limb widths. The packaged test command covers the first five values. The additional $p = 4096$ check is a historical check outside this package. This work only calculates the remainder. It does not decide whether the number is prime.

2.6 Difference

It works for one large modulus, $2^p - 1$, instead of seven small primes. It is like a single ring whose dial has $2^p - 1$ teeth.

3. Folding gate and exponent-positional shadow

3.1 Date and source

9 September 2026. Channel model and register model from the Mersenne research line.

3.2 Idea

The remainder modulo a small prime can also be found by binary folding instead of octal digit flow. For each channel, a w is chosen such that $2^w \equiv \pm 1 \pmod{q}$. Then the bottom w bits of the number and the upper part are added or subtracted, and the remainder does not change.

3.3 Mechanism

q	w	Identity	Folding
3	**2**	$2^2 \equiv +1$	$LOW + HIGH$
7	**3**	$2^3 \equiv +1$	$LOW + HIGH$
5	**2**	$2^2 \equiv -1$	$LOW - HIGH$
17	**4**	$2^4 \equiv -1$	$LOW - HIGH$
11	**5**	$2^5 \equiv -1$	$LOW - HIGH$
13	**6**	$2^6 \equiv -1$	$LOW - HIGH$

The value is folded until it fits in w bits. Then q is added or subtracted at most a few times to bring it into the range $0 \le r < q$. Channel 2 needs no folding: the remainder is the last bit, and the exponent is the number of trailing zero bits.

3.4 Example

738**, in channel 7:** $738 \to 2 + 92 = 94 \to 6 + 11 = 17 \to 1 + 2 = 3$**. In channel 11: the bottom 5 bits are** 2**, the upper part is** 23**,** $2 - 23 = -21$**, and adding 11 twice gives** 1**. Indeed,** $738 \bmod 7 = 3$**,** $738 \bmod 11 = 1$**.**

3.5 Code and testing

Python remainder gate. For six channels and every input in $0..65535$**, 393,216 results were tested, with no errors. I also wrote the same folding separately and tried it over the same range here. Every result was correct.**

3.6 Exponent-positional shadow

The same study also defined a shadow candidate that spreads the octal digits of a large exponent across rows: $E(q) = e_1(q) + 8\,e_2(q) + 64\,e_3(q) + 512\,e_4(q)$**. For example, for** 3^{14}**,** $E_3 = 16_8$**, that is,** $e_1 = 6$**,** $e_2 = 1$**. The flowing counter became the working form of this candidate.**

3.7 Difference

Binary folding is used to find the remainder instead of octal digit flow.

4. Digit weights and cutting

4.1 Date and source

20 July 2026, updated on 2 August 2026. Shadow mechanism diagram and simple descent card.

4.2 Idea

A number's remainder modulo a prime is the remainder of the sum of its octal digits with fixed weights modulo that prime. The weights are the sequence $8^k \bmod q$ and form a short cycle for each prime. The descent of the rough value is not a subtraction and a division. It is cutting off the bottom three bits and taking the rest.

4.3 Mechanism

The weights, starting from the lowest digit:

q	**Weights** ($8^k \bmod q$)
3	**1, 2**
5	**1, 3, 4, 2**
7	**1 (digit sum)**
11	**1, 8, 9, 6, 4, 10, 3, 2, 5, 7**
13	**1, 8, 12, 5**
17	**1, 8, 13, 2, 16, 9, 4, 15**

Channel 2 is a shift and does not enter the table. The remainder is $r_q = \sum_k d_k \cdot w_k \bmod q$.

4.3.1 Cutting

$\mathrm{rough} - (\mathrm{rough} \,\&\, 7)$ **followed by** $/8$ **means cutting off the bottom three bits and taking the rest. Neither a subtractor nor a divider is needed. It is only wiring in the circuit. The three cut bits are written as they are into the next row's a cell. The remaining rough is carried in a rolling rough register that is overwritten at each step.**

4.4 Example

$360 = 550_8$**, digits from the lowest are** $0, 5, 5$**. In channel 11,** $0 \cdot 1 + 5 \cdot 8 + 5 \cdot 9 = 85$ **and** $85 \bmod 11 = 8$**. In channel 13,** $0 + 40 + 60 = 100$ **and** $100 \bmod 13 = 9$**. Indeed,** $360 \bmod 11 = 8$, $360 \bmod 13 = 9$**. Cutting:** $59 = 73_8$**, the bottom three bits are** 3**, the rest is** 7**.**

4.5 Code and testing

According to the recorded measurement, the weights were verified for every number up to 2^{24}, with no errors. The six channels for 360 were also calculated separately here, and all were correct.

4.6 Difference

In later forms, the same remainder was found with the Horner form as the digits flowed from left to right. Here the digits are summed at the same time with fixed weights.

5. First version: two-beat pipeline

5.1 Date and source

30 June 2026. The first full analysis of the Resolver unit. Its abstract was rewritten on 3 August and 1 September 2026.

5.2 Idea

There is no general divider, but Resolver always divides by the same seven fixed primes. Division by a fixed number can be done with shifts and addition. The divisibility test is cheaper still.

5.3 Mechanism

There are two beats.

5.3.1 First beat, divisibility

Eight octal digits flow from left to right, and the remainders of seven channels are updated at the same time for each digit: $r_q \leftarrow (8\,r_q + d) \bmod q$**. The intermediate value is at most** $8q - 1$**, and the remainder is found by comparing with and subtracting** q**. The last digit alone only tells divisibility by 2, 4 and 8. Odd primes need all the digits.**

5.3.2 Second beat, finding exponents

For primes that divide the number, the exponent is found with a fixed q **division network and a counter: the number is divided until it is no longer divisible, and the counter gives the exponent. The exponent of 2 is read in one step as the number of trailing zero bits.**

The residual not divisible by the seven primes is written in plain form to rc **and is not opened. In this version, Resolver ran on every load. In later versions, the shadow became a lazy shadow built at the first multiplicative operation.**

5.4 Example

$510510 = 2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 = 01745056_8$**. After eight digits have flowed through, the remainder in all seven channels is 0.** 360**: divided by 2 three times, by 3 twice, by 5 once, shadow** $[3, 2, 1, 0, 0, 0, 0]$**, residual 1.**

5.5 Code and testing

Recorded measurement in the old emulator: about 117 million divisibility checks for all 24-bit numbers and seven primes, and 16.7 million conversions and reverse conversions, with no errors. Finding exponents took an average of 1.66 steps, with a worst case of 15 steps (3^{15}**). In the emulator source, Test 7 covers** $1 \le N \le 2^{24} - 1$**. Each successful division by an odd channel prime counts as one step, and all factors of 2 together count as one step when present. Octal digit flow and unsuccessful divisibility checks are not included. I reran this emulator on 25 October 2026 and reproduced these counts. They count software-model steps, not measured clock cycles.**

5.6 Difference

This first version carries the residual in plain form. The idea of taking the residual down by dividing it by eight was described here but was not implemented in code. All later forms were built on these two beats.

6. Open questions

It has not been settled whether shadow rows will be the octal digits of one exponent, as in the flowing counter, or levels of the ladder, as in the layered ladder. Whether rc holds the first rough factor or the last unprocessed residual is open. Zero, negative and fractional inputs must be defined separately.

FPGA experiments will be done on these forms. I will share their results separately.

Related pages: Resolver experiments, Gear model, Register and Parts.

James Augustus, 2026. Licensed under CC BY 4.0. https://github.com/JamesAugustus/ctcpu-hunter